Managing Traffic with OpenShift Service Mesh

Lesson | Lab Environment

Fault Injection with OpenShift Service Mesh

Objectives

- Explain the purpose of fault injection in service meshes.
- Configure delay faults to simulate slow backends.
- Configure abort faults to simulate service errors.
- Discuss best practices for safely using fault injection.

Fault Injection with OpenShift Service Mesh

In distributed systems, networks can fail and latency can spike, leading to unpredictable application behavior. To build resilient applications, you must test how your services respond to these conditions.

Chaos engineering is the practice of intentionally injecting failures into a system to test its response. By simulating conditions such as network latency and service unavailability, you can identify and fix weaknesses before they impact production users.

Red Hat OpenShift Service Mesh provides fault injection capabilities that enable controlled chaos testing without modifying application code. OpenShift Service Mesh supports two fault types through the `HTTPFaultInjection` configuration object in virtual service resources:

Delays
Simulate network latency or overloaded services by adding artificial delays to requests. This tests timeout and retry configurations.

Aborts
Simulate service failures by terminating requests with specific HTTP or gRPC error codes. This validates error handling, fallback mechanisms, and circuit breaker logic.

Configuring Delays

To test application behavior with slow dependencies, configure delay injection by using the `HTTPFaultInjection.Delay` object within a virtual service resource.

The `Delay` object provides the following main parameters:

`percentage`
The percentage of requests that receive the delay. Each request has an independent probability of receiving the fault.

`fixedDelay`
The delay duration, such as `5s` for 5 seconds.

The following virtual service example configures a 5-second delay applied to 10% of the requests.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
spec:
  hosts:
  - example-svc
  http:
  - route:
    - destination:
        host: example-svc
        subset: v1
    fault: (1)
      delay: (2)
        percentage: (3)
          value: 10.0
        fixedDelay: 5s (4)
```

(1) The `HTTPFaultInjection` configuration object for injecting faults.

(2) The `HTTPFaultInjection.Delay` configuration object for delay injection.

(3) Percentage of requests to delay.

(4) Delay duration.

Configuring Aborts

To test application error handling, configure abort injection by using the `HTTPFaultInjection.Abort` object within a virtual service resource.

The `Abort` object provides the following main parameters:

`percentage`

The percentage of requests to abort. Each request has an independent probability of receiving the fault.

`httpStatus`
The HTTP status code to return, such as `503`.

`grpcStatus`
The gRPC status code to return, such as `UNAVAILABLE`. Use this parameter instead of `httpStatus` for gRPC services.

The following virtual service configures an abort of 20% of the requests with an HTTP 503 status code.

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: example-vs
spec:
  hosts:
  - example-svc
  http:
  - route:
    - destination:
        host: example-svc
        subset: v1
    fault: (1)
      abort: (2)
        percentage: (3)
          value: 20.0
        httpStatus: 503 (4)
```

(1) The `HTTPFaultInjection` configuration object for injecting faults.

(2) The `HTTPFaultInjection.Abort` configuration object for error injection.

(3) Percentage of requests to abort.

(4) The HTTP status code to return.

Using Fault Injection Safely and Effectively

Fault injection is useful for validating application resiliency in the following scenarios:

Simulating partial failures
Test service behavior when dependencies are slow or unavailable to fine-tune the application response to partial degradation.

Validating client-side logic
Verify that resiliency policies, such as retries and timeouts, work as expected when a downstream service misbehaves.

Testing circuit breakers
Trigger circuit breakers by injecting sustained errors to ensure that they open correctly and prevent cascading failures.

Staging environment validation
Safely replicate production faults in staging to debug issues without affecting users.

To use fault injection safely and effectively, consider the following guidelines:

Start small
Limit the breadth of damage by applying faults to a small percentage of traffic or targeting specific test users.

Monitor everything
Use observability tools to measure fault impact. Observe latency changes, error rates, and traffic distribution patterns. Verify that monitoring and alerting systems detect injected faults as expected.

Automate cleanup
Automatically remove fault injection rules after tests complete.

Test in a representative environment
Test environments closer to production yield more valuable results.

REFERENCES

Istio.io: Fault Injection (https://istio.io/v1.26/docs/tasks/traffic-management/fault-injection/)

Istio.io: HTTPFaultInjection Delay (https://istio.io/v1.26/docs/reference/config/networking/virtual-service/#HTTPFaultInjection-Delay)

Istio.io: HTTPFaultInjection Abort (https://istio.io/v1.26/docs/reference/config/networking/virtual-service/#HTTPFaultInjection-Abort)

Wikipedia: Fallacies of distributed computing (https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing)

Wikipedia: Chaos engineering (https://en.wikipedia.org/wiki/Chaos_engineering)

Revision: ad0008l-3.1-ed4-20260310-8ae9760

이용 약관 (https://www.redhat.com/ko/about/terms-use)

모든 정책 및 지침 (https://www.redhat.com/ko/about/all-policies-guidelines)

릴리스 노트 (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

쿠키 기본설정